Helion Hackathon

PyTorch

Presented by

Meta NVIDIA

Clement Anthonioz Blanc

Meta, AI Partnerships

James Choi

Meta, PyTorch

Oguz Ulgen

Meta, Principal Software Engineer

Helion: A High-level DSL for Kernel Authoring

Oguz Ulgen

PyTorch @ Meta MSL

March 14th, 2026

What’s Helion?

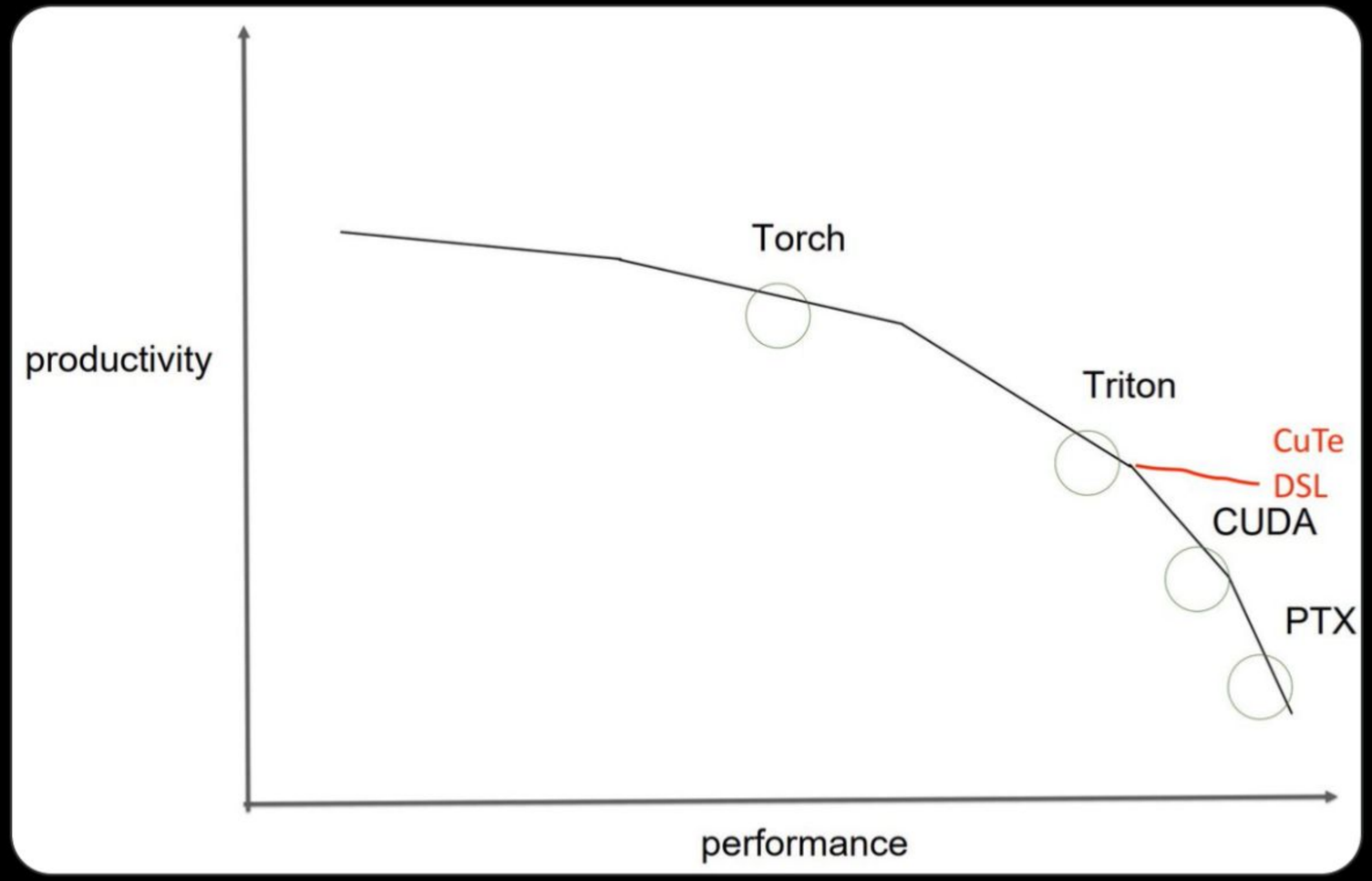
- Helion lets you author tile-based kernels using Pytorch
  - No more passing tensor sizes & strides
- It lowers to Triton (soon: Pallas, later: CuTe DSL)
  - One helion kernel corresponds to many triton kernels
- It comes with a powerful autotuner
  - no more sprinkling eviction hints or stage counts through your code

- In effect, this makes it easier to write kernels than Triton
- With potentially better performance due to better tuning
- Removes tuning knobs as a “free variable,” and systematically tests instead

Tri Dao @tri_dao · Jul 10, 2025

I really like Phil Tillet's framing of different tools having different tradeoffs in productivity and performance: torch compile, triton, CUDA, PTX. It's still early but CuTe-DSL and similar Python-based DSL might bend this curve. And soon we can probably get LLMs to generate these kernels!

Phil's talk here: semianalysis.com/wp-content/upl...



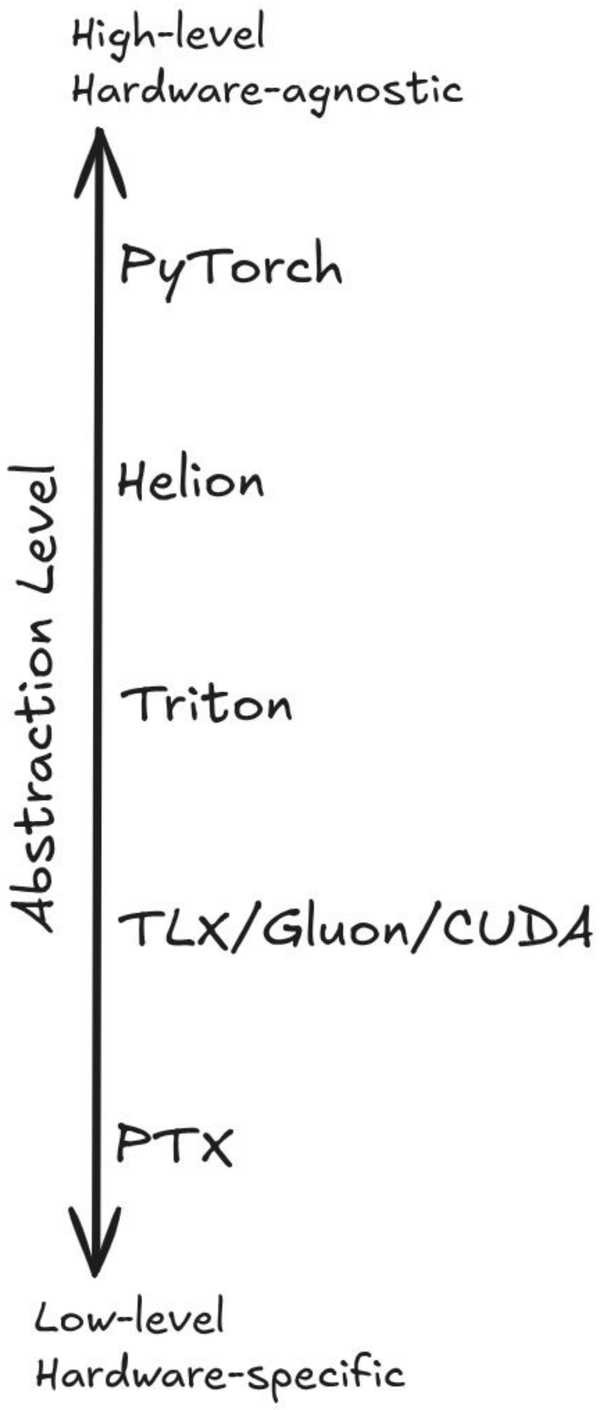
Tri Dao @tri_dao · Jul 10, 2025

I really like Phil Tillet's framing of different tools having different tradeoffs in productivity and performance: torch compile, triton, CUDA, PTX. It's still early but CuTe-DSL and similar Python-based DSL might bend this curve. And soon we can probably get LLMs to generate these kernels!

Phil's talk here: semianalysis.com/wp-content/upl...

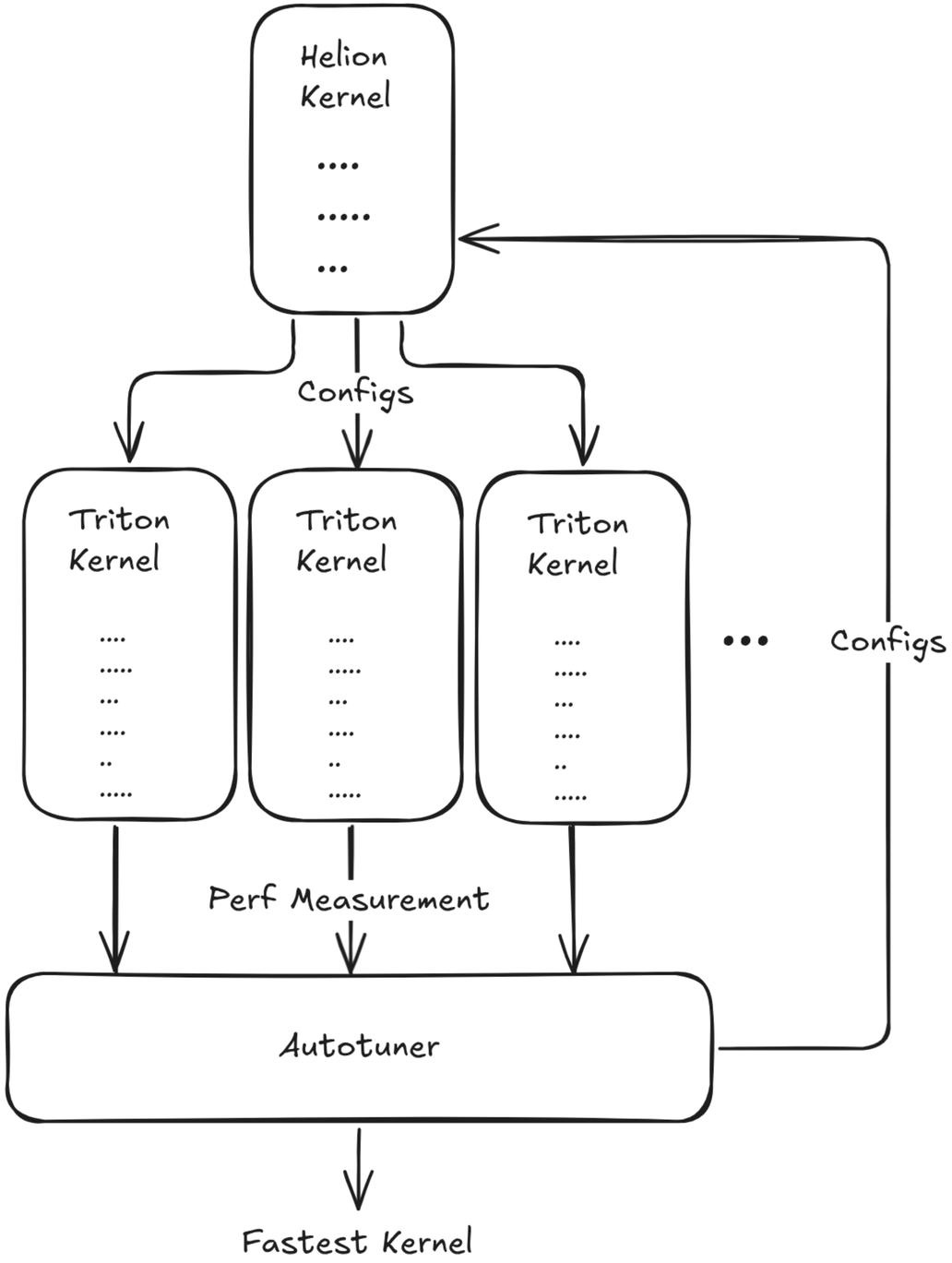
Motivation: Why a new DSL?

- Thousands of custom kernels used at large companies
  - Custom kernels turn into tech debt quickly
  - Hard to port to new hardware
  - Heterogeneous hardware is a new trend
- Higher level: automate things that are manual in Triton:
  - Tensor indexing (strides, block_ptr/pointer/TMA)
  - Wrapper/grid/block sizes definitions
  - Defining search spaces
  - Arg management
  - Templating
- Provides more control than *torch.compile*
- Better performance portability
- Autotuning (see next slide)



Autotuning

- One Helion kernels maps to many Triton kernels, defined by a config
- Search spaces, are created implicitly
  - E.g. hl.tile([m, n]) implies block sizes, iteration order, flattening, swizzling, etc choices
- The autotuner can search thousands of configurations (each corresponding to a Triton kernel)
- Autotuning is done once ahead-of-time, and resulting tuned configuration are deployed



Hardware Heterogeneity

Helion currently supports various hardwares via Triton backend:

- Nvidia A100, H100, B200, GB200+
- AMD MI300X+
- CPU (via Triton CPU)
- Accelerators that can consume Triton

Soon:

- Google TPUs

Matrix Multiply Example

```python
import torch, helion, helion.language as hl

@helion.kernel()
def matmul(x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    m, k = x.size()
    k, n = y.size()
    out = torch.empty([m, n], dtype=x.dtype, device=x.device)

    for tile_m, tile_n in hl.tile([m, n]):
        acc = hl.zeros([tile_m, tile_n], dtype=torch.float32)
        for tile_k in hl.tile(k):
            acc = torch.addmm(acc, x[tile_m, tile_k], y[tile_k, tile_n])
        out[tile_m, tile_n] = acc

    return out
```

Language Constructs

- hl.tile(sizes) subdivides iteration space into tiles
  - Tile sizes, iteration order, flattening, swizzling is autotuned
  - Outermost hl.tile becomes GPU launch grid
  - Inner hl.tile become loops on device
- Standard PyTorch ops can be used
  - Familiarity with PyTorch means you already know most of Helion
  - Pointwise ops (add, sigmoid, etc): lowered with TorchInductor
  - Reduction ops (sum, softmax, etc): lowered with TorchInductor
  - Matmul ops: become tl.dot
  - View ops: become tl.* ops
- Control flow
  - Supported in the top level kernel (becomes multiple FX graphs in Helion IR)
  - Function calls are traced with make_fx

Configuration Space

Indexing Choices

Helion

v = x[tile0, tile1]

Output Triton (Helion config.indexing = "pointer")

```
indices_0 = pid_0 * _BLOCK_SIZE_0 + tl.arange(0, _BLOCK_SIZE_0)
mask_0 = indices_0 < x_size_0
…
v = tl.load(x + indices_0[:, None] * x_stride_0
                + indices_1[None, :] * x_stride_1,
            mask_0[:, None] & mask_1[None, :], other=0)
```

Output Triton (Helion config.indexing = "block_ptr")

```
v = tl.load(tl.make_block_ptr(x,
    [x_size_0, x_size_1], [x_stride_0, x_stride_1], [offset_0, offset_1],
    [_BLOCK_SIZE_0, _BLOCK_SIZE_1], [1, 0]),
    boundary_check=[0, 1], padding_option='zero'))
```

Output Triton (Helion config.indexing = "tensor_descriptor")

```
x_desc = tl.make_tensor_descriptor(x, [x_size_0, x_size_1], [x_stride_0,
x_stride_1], [_BLOCK_SIZE_0, _BLOCK_SIZE_1])
…
v = x_desc.load([offset_0, offset_1])
```

Implicit Block Sizes

Helion

for tile0, tile1 in hl.tile([x, y]):

Output Triton (Helion config.block_sizes = [32, 32])

```
@triton.jit
def _add_kernel(...,
                _BLOCK_SIZE_0: tl.constexpr,
                _BLOCK_SIZE_1: tl.constexpr):
    num_blocks_0 = tl.cdiv(x_size_0, _BLOCK_SIZE_0)
    pid_0 = tl.program_id(0) % num_blocks_0
    pid_1 = tl.program_id(0) // num_blocks_0
    offset_0 = pid_0 * _BLOCK_SIZE_0
    indices_0 = (offset_0 + tl.arange(0, _BLOCK_SIZE_0)).to(tl.int32)
    mask_0 = indices_0 < x_size_0
    offset_1 = pid_1 * _BLOCK_SIZE_1
    indices_1 = (offset_1 + tl.arange(0, _BLOCK_SIZE_1)).to(tl.int32)
    …

# Call the kernel:
_BLOCK_SIZE_0 = 32
_BLOCK_SIZE_1 = 32
_add_kernel[triton.cdiv(x.size(0), _BLOCK_SIZE_0) *
            triton.cdiv(x.size(1), _BLOCK_SIZE_1)](
…_BLOCK_SIZE_0, _BLOCK_SIZE_1, …)
```

Loop Flattening

Helion

for tile0, tile1 in hl.tile([x, y])

Output Triton (Helion config.flatten_loops = [True])

```
@triton.jit
def _add_kernel(..., _BLOCK_SIZE_0_1: tl.constexpr):
    offsets_0_1 = tl.program_id(0) * _BLOCK_SIZE_0_1 + tl.arange(
            0, _BLOCK_SIZE_0_1)
    indices_1 = offsets_0_1 % x_size_1
    indices_0 = offsets_0_1 // x_size_1
    mask_0_1 = offsets_0_1 < x_size_0 * x_size_1
    …
```

```
# Call the kernel
_BLOCK_SIZE_0_1 = 1024
_add_kernel[triton.cdiv(x.size(0) * x.size(1), _BLOCK_SIZE_0_1)](
    …, _BLOCK_SIZE_0_1, …)
```

Reduction Rolling

Helion

```
v = x[tile0, :].sum(1)
```

Output Triton (Helion config.reduction_loops = None)

```
load = tl.load(x, …)  # loads entire row
v = tl.sum(load, 1)
```

Output Triton (Helion config.reduction_loops = [32])

```
acc = tl.full([_BLOCK_SIZE_0, _REDUCTION_BLOCK_1], 0)
for roffset_1 in tl.range(0, x_size_1, _REDUCTION_BLOCK_1):
    rindex_1 = roffset_1 + tl.arange(0, _REDUCTION_BLOCK_1)
    mask_1 = rindex_1 < x_size_1
    load = tl.load(x + …)
    acc += load
v = tl.sum(sum_1_acc, 1)
```

PID Type (Persistent Kernels)

Output Triton (Helion config.pid_type = "flat")

```
@triton.jit
def kernel(...):
    …

kernel[triton.cdiv(x.size(0), _BLOCK_SIZE_0) *
       triton.cdiv(x.size(1), _BLOCK_SIZE_1)](...)
```

Output Triton (Helion config.pid_type = "xyz")

```
@triton.jit
def kernel(...):
    …

kernel[triton.cdiv(x.size(0), _BLOCK_SIZE_0),
       triton.cdiv(x.size(1), _BLOCK_SIZE_1)](...)
```

Output Triton (Helion config.pid_type = "persistent_interleaved")

```
@triton.jit
def kernel(...):
    total_pids = (tl.cdiv(x_size_0, _BLOCK_SIZE_0) *
                  tl.cdiv(x_size_1, _BLOCK_SIZE_1))
    for virtual_pid in tl.range(tl.program_id(0), total_pids, _NUM_SM):
        …

kernel[_NUM_SM](...)
```

Output Triton (Helion config.pid_type = "persistent_blocked")

```
@triton.jit
def kernel(...):
    total_pids = (tl.cdiv(x_size_0, _BLOCK_SIZE_0) *
                  tl.cdiv(x_size_1, _BLOCK_SIZE_1))
    block_size = tl.cdiv(total_pids, _NUM_SM)
    start_pid = tl.program_id(0) * block_size
    end_pid = tl.minimum(start_pid + block_size, total_pids)
    for virtual_pid in tl.range(start_pid, end_pid):
        …

kernel[_NUM_SM](...)
```

L2 Grouping / Loop Reordering

Output Triton (Helion config.l2_grouping = None)

```
num_blocks_0 = tl.cdiv(x_size_0, _BLOCK_SIZE_0)
pid_0 = tl.program_id(0) % num_blocks_0
pid_1 = tl.program_id(0) // num_blocks_0
```

Output Triton (Helion config.loop_orders = [[1, 0]])

```
num_blocks_0 = tl.cdiv(x_size_1, _BLOCK_SIZE_1)
pid_0 = tl.program_id(0) % num_blocks_1
pid_1 = tl.program_id(0) // num_blocks_1
```

… (usage of pid_0/pid_1 swapped throughout code)

Output Triton (Helion config.l2_grouping = 32)

```
num_pid_m = tl.cdiv(x_size_0, _BLOCK_SIZE_0)
num_pid_n = tl.cdiv(x_size_1, _BLOCK_SIZE_1)
inner_2d_pid = tl.program_id(0)
num_pid_in_group = 32 * num_pid_n
group_id = inner_2d_pid // num_pid_in_group
first_pid_m = group_id * 32
group_size_m = min(num_pid_m - first_pid_m, 32)
pid_0 = (first_pid_m + inner_2d_pid %
            num_pid_in_group % group_size_m)
pid_1 = inner_2d_pid % num_pid_in_group // group_size_m
```

Triton Tunables

```
config.range_unroll_factors = [2]
config.range_warp_specializes = [True]
config.range_num_stages = [3]
config.range_multi_buffers = [False]
config.range_flattens = [True]
```

Output Triton

```
for offset_1 in tl.range(
    0, x_size_1.to(tl.int32), _BLOCK_SIZE_1,
    loop_unroll_factor=2,
    warp_specialize=True,
    num_stages=3,
    disallow_acc_multi_buffer=True,
    flatten=True,
):
```

```
config.num_warps = 16
config.num_stages = 4
```

```
kernel[...](..., num_warps=16, num_stages=4)
```

Existing Triton tunables are automatically explored by Helion autotuner

Automatic Masking

Helion

v = (x+1).sum(1)

Output Triton (Helion settings.static_shapes = False)

```
tmp0 = x + 1
tmp1 = tl.where(mask_0[:, None] & mask_1[None, :], tmp0, 0)
v = tl.sum(tmp1, 1)
```

Output Triton (Helion settings.static_shapes = True + divisible sizes)

```
tmp0 = x + 1
v = tl.sum(tmp0, 1)
```

`x+1` turns zero masked elements coming from the load into ones, so Helion automatically inserts an extra `tl.where` mask.
This is easy to get wrong in Triton.

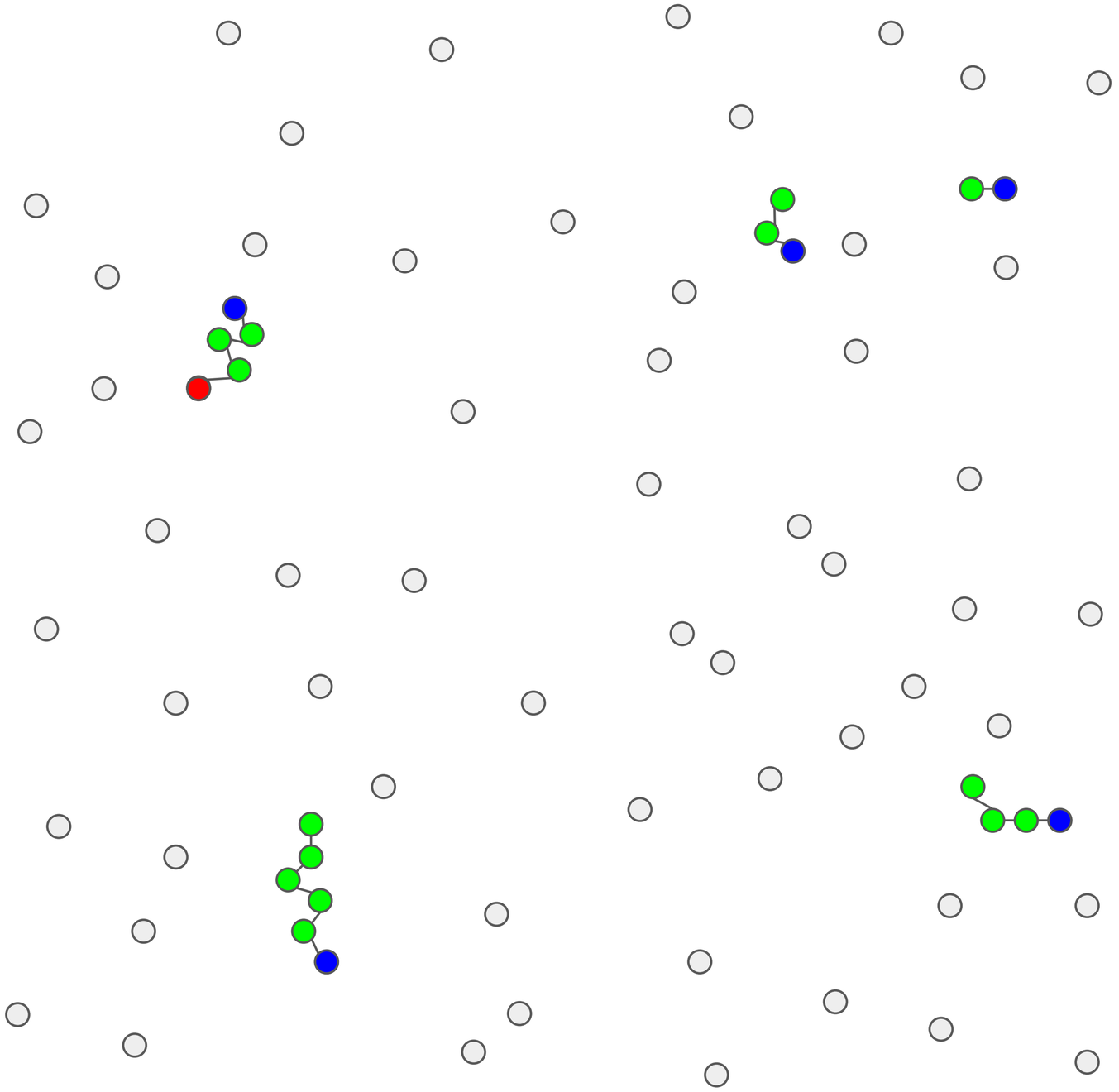
Autotuning

Autotuning Algorithms

- Takes about 10 minutes to search 1200 candidate Triton kernels
- Today:
  - Pattern search
  - Differential evolution
  - Finite search
  - DE-Surrogate hybrid (OSS contribution: https://github.com/pytorch/helion/pull/1096)
  - (default) LFBO pattern search (OSS contribution: https://github.com/pytorch/helion/pull/1115)
- In the future:
  - LLM-guided search
  - Reinforcement learning
  - Shared performance databases
  - Other search techniques

Pattern search

1. Initial population of 100 random configs
2. Select top 5 fastest
3. Hill-climb to local minimums
4. Select fastest



LFBO Pattern Search (Default)

- Likelihood-Free Bayesian Optimization (LFBO) Pattern Search
- Enhances PatternSearch by using a Random Forest classifier as a surrogate model to select which configurations to benchmark

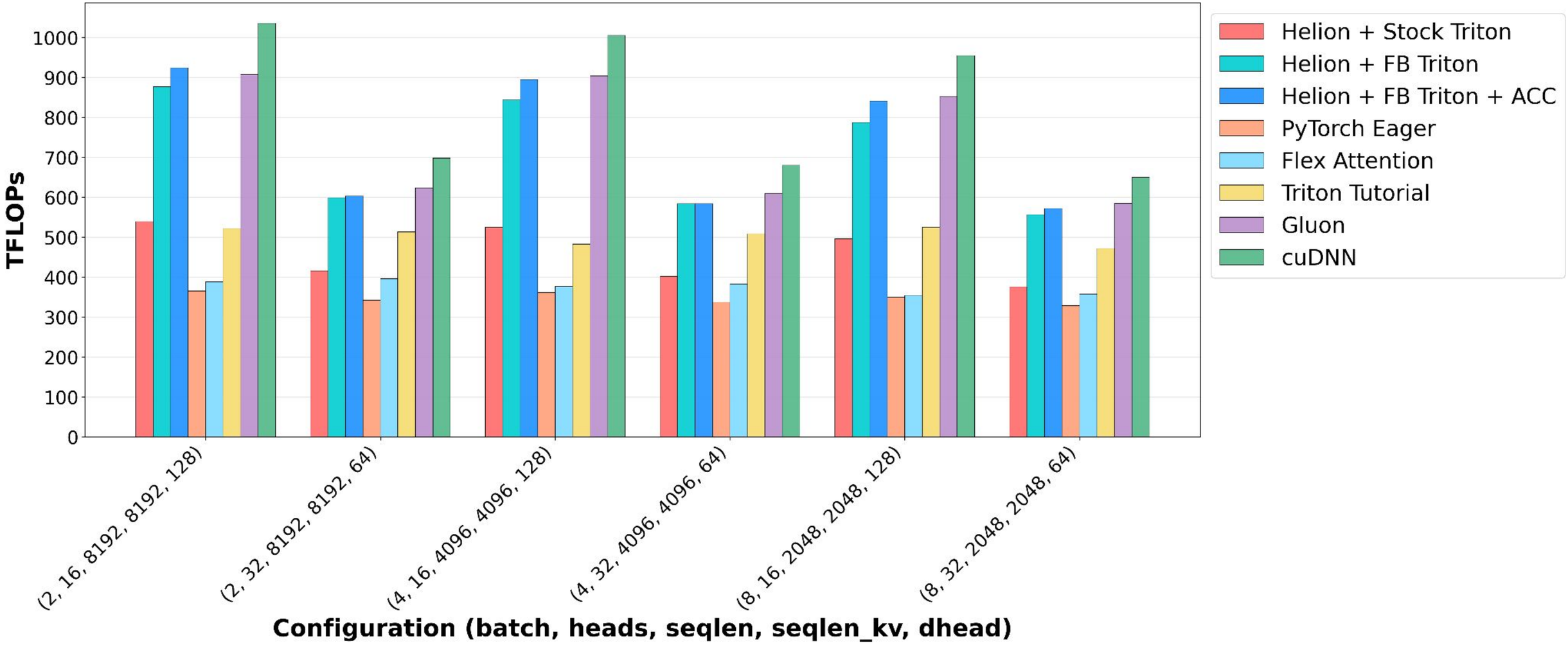
Deploying a single autotuned configuration

```
@helion.kernel(config=helion.Config(
    block_sizes=[64, 64, 64],
    loop_orders=[[0, 1]],
    l2_groupings=[4],
    range_unroll_factors=[0, 1],
    range_warp_specializes=[None, False],
    range_num_stages=[0, 3],
    range_multi_buffers=[None, False],
    range_flattens=[None, None],
    num_warps=8,
    num_stages=6,
    indexing='block_ptr',
    pid_type='flat'
))
def matmul(x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
```

Autotuning should be done ahead of time in a development/staging environment

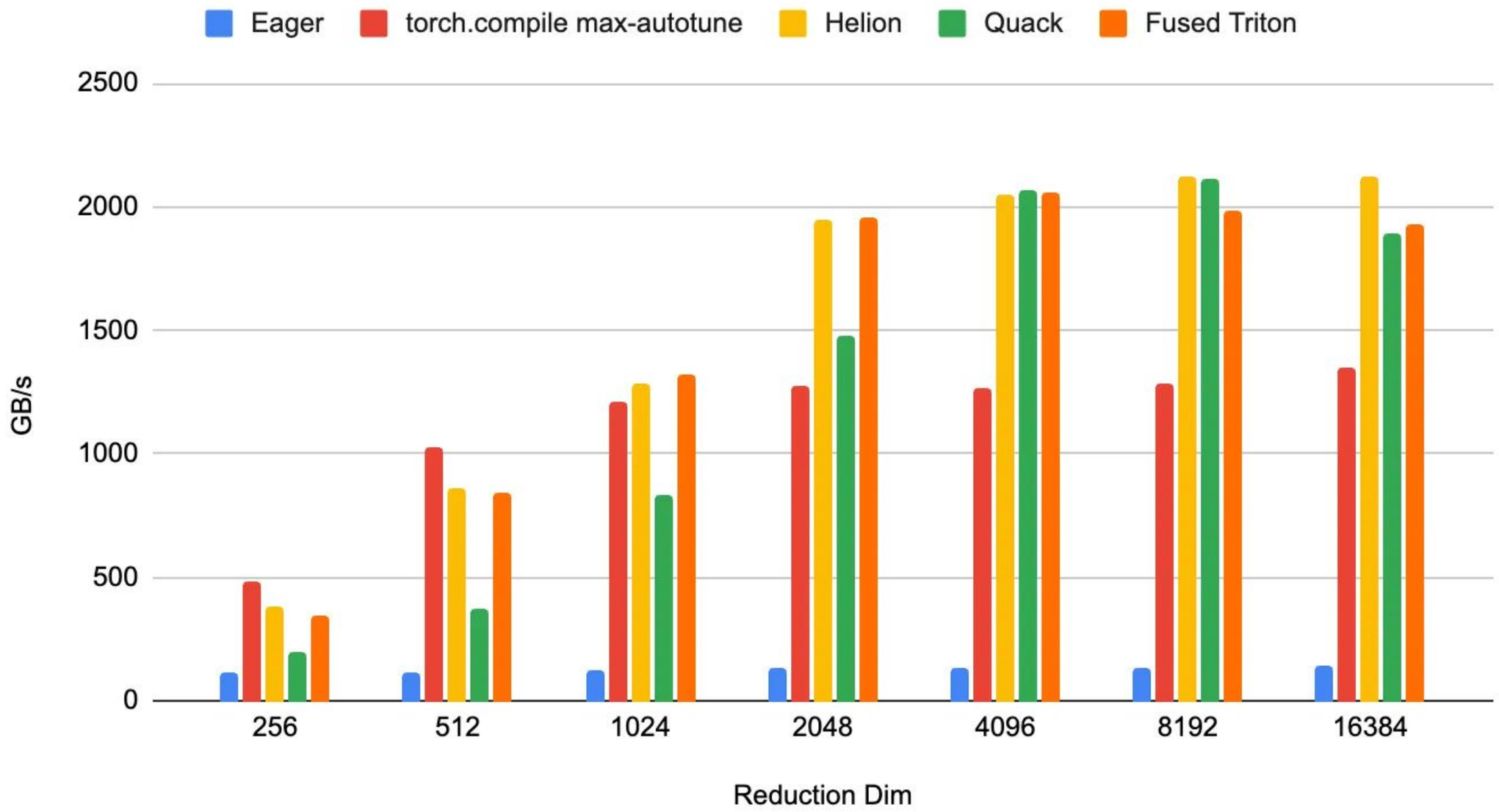
Performance

Case Study 1: Attention Performance Comparison (NVIDIA B200 700W)



FB Triton branch: https://github.com/facebookexperimental/triton/tree/ws-3.5
ACC: Advanced Compiler Configuration for PTXAS

Case Study 3: Comparison to CuTeDSL/Quack
For RMSNorm backwards (M=32768, NVIDIA H100)



NVIDIA B200 Speedups Over Eager PyTorch

Kernel	Helion Speedup	Torch Compile Speedup	Triton Speedup
cross_entropy	3.52	2.18	1.09
gemm	0.85	0.69	0.70
int4_gemm	7.53	8.89	3.23
jsd	8.89	8.99	1.43
kl_div	3.80	3.25	3.76
layer_norm	1.92	1.69	1.47
layer_norm-bwd	1.54	1.12	0.70
rms_norm	5.78	5.73	5.07
rms_norm-bwd	4.97	4.17	3.28
softmax	3.49	1.53	2.84
welford	2.00	2.04	0.79
geomean	**3.27**	**2.70**	**1.76**

AMD MI350X Speedups Over PyTorch Eager

Kernel	Helion Speedup	Torch Compile Speedup	Triton Speedup
geglu	1.01	1.01	1.01
gemm (fp16)	1.17	1.27	1.21
int4_gemm	7.58	5.98	1.68
jsd	12.07	12.06	2.75
kl_div	3.69	3.63	3.44
layer_norm	1.18	1.11	1.10
rms_norm	3.11	3.00	3.18
softmax	1.58	1.32	1.30
swiglu	1.04	1.04	1.04
geomean	**2.37**	**2.26**	**1.65**

How to debug a Helion kernel?

```
@helion.kernel
def rms_norm(x: torch.Tensor, weight: torch.Tensor, eps: float = 1e-5):
    m, n = x.size()
    out = torch.empty_like(x)
    inv_rms = torch.empty([m], dtype=x.dtype, device=x.device)
    for tile_m in hl.tile(m):
        x_tile = x[tile_m, :].to(torch.float32)
        x_squared = x_tile * x_tile
        mean_x_squared = torch.mean(x_squared, dim=-1)
        inv_rms_tile = torch.rsqrt(mean_x_squared + eps)
        normalized = x_tile * inv_rms_tile[:, None]
        out[tile_m, :] = (normalized * weight[:].to(torch.float32)).to(out.dtype)
        inv_rms[tile_m] = inv_rms_tile.to(out.dtype)
    return out, inv_rms.reshape(-1, 1)
```

Step 1: Use **interpret mode**, to iterate and make sure algorithm is correct

```
@helion.kernel
def rms_norm(x: torch.Tensor, weight: torch.Tensor, eps: float = 1e-5):
    m, n = x.size()
    out = torch.empty_like(x)
    inv_rms = torch.empty([m], dtype=x.dtype, device=x.device)
    for tile_m in hl.tile(m):
        x_tile = x[tile_m, :].to(torch.float32)
        print("x_tile: ", x_tile)
        x_squared = x_tile * x_tile
        mean_x_squared = torch.mean(x_squared, dim=-1)
        inv_rms_tile = torch.rsqrt(mean_x_squared + eps)
        normalized = x_tile * inv_rms_tile[:, None]
        out[tile_m, :] = (normalized * weight[:].to(torch.float32)).to(out.dtype)
        inv_rms[tile_m] = inv_rms_tile.to(out.dtype)
    return out, inv_rms.reshape(-1, 1)
```

Can use `print()` or `breakpoint()` in interpret mode!

```
$ HELION_INTERPRET=1 python rms_norm.py

x_tile:  tensor([[0.8560,  0.4290,  0.4236,  ...,]]])
...
```

Step 2: Run **compile mode without autotuning (`effort=none`)** to verify it works

```
@helion.kernel
def rms_norm(x: torch.Tensor, weight: torch.Tensor, eps: float = 1e-5):
    m, n = x.size()
    out = torch.empty_like(x)
    inv_rms = torch.empty([m], dtype=x.dtype, device=x.device)
    for tile_m in hl.tile(m):
        x_tile = x[tile_m, :].to(torch.float32)
        x_squared = x_tile * x_tile
        mean_x_squared = torch.mean(x_squared, dim=-1)
        inv_rms_tile = torch.rsqrt(mean_x_squared + eps)
        normalized = x_tile * inv_rms_tile[:, None]
        out[tile_m, :] = (normalized * weight[:].to(torch.float32)).to(out.dtype)
        inv_rms[tile_m] = inv_rms_tile.to(out.dtype
    return out, inv_rms.reshape(-1, 1)
```

`HELION_AUTOTUNE_EFFORT=none` makes it use default config

```
$ HELION_AUTOTUNE_EFFORT=none HELION_INTERPRET=0 python rms_norm.py

Using default config: @helion.kernel(config=helion.Config(...), ...)
...
```

Step 3: Run **compile mode with `effort="quick"`** to get some autotuning result

```python
@helion.kernel
def rms_norm(x: torch.Tensor, weight: torch.Tensor, eps: float = 1e-5):
    m, n = x.size()
    out = torch.empty_like(x)
    inv_rms = torch.empty([m], dtype=x.dtype, device=x.device)
    for tile_m in hl.tile(m):
        x_tile = x[tile_m, :].to(torch.float32)
        x_squared = x_tile * x_tile
        mean_x_squared = torch.mean(x_squared, dim=-1)
        inv_rms_tile = torch.rsqrt(mean_x_squared + eps)
        normalized = x_tile * inv_rms_tile[:, None]
        out[tile_m, :] = (normalized * weight[:].to(torch.float32)).to(out.dtype)
        inv_rms[tile_m] = inv_rms_tile.to(out.dtype)
    return out, inv_rms.reshape(-1, 1)
```

```
$ HELION_AUTOTUNE_EFFORT=quick HELION_INTERPRET=0 python rms_norm.py

[0s] Starting autotuning process, this may take a while...
...
[69s] Autotuning complete in 69.6s after searching 290 configs.
One can hardcode the best config and skip autotuning with:
    @helion.kernel(config=helion.Config(...), ...)
```

Found a better config after some autotuning!

Step 4: Run **compile mode with effort="full"** to get the best autotuning result

```
@helion.kernel
def rms_norm(x: torch.Tensor, weight: torch.Tensor, eps: float = 1e-5):
    m, n = x.size()
    out = torch.empty_like(x)
    inv_rms = torch.empty([m], dtype=x.dtype, device=x.device)
    for tile_m in hl.tile(m):
        x_tile = x[tile_m, :].to(torch.float32)
        x_squared = x_tile * x_tile
        mean_x_squared = torch.mean(x_squared, dim=-1)
        inv_rms_tile = torch.rsqrt(mean_x_squared + eps)
        normalized = x_tile * inv_rms_tile[:, None]
        out[tile_m, :] = (normalized * weight[:].to(torch.float32)).to(out.dtype)
        inv_rms[tile_m] = inv_rms_tile.to(out.dtype)
    return out, inv_rms.reshape(-1, 1)
```

```
$ HELION_AUTOTUNE_EFFORT=full HELION_INTERPRET=0 python rms_norm.py

[0s] Starting autotuning process, this may take a while...
...
[524s] Autotuning complete in 524.2s after searching 1971 configs.
One can hardcode the best config and skip autotuning with:
    @helion.kernel(config=helion.Config(...), ...)
```

Found the best config!

Kernel Development Workflow

- `HELION_INTERPRET=1`, check ✓ algorithm is correct ✓ passes numerical check
  - Can use `print()` or `breakpoint()`
  - Interpret mode works => compile mode works
- `HELION_AUTOTUNE_EFFORT=`**`none`**, check ✓ compile mode can run
- `HELION_AUTOTUNE_EFFORT=`**`quick`**, do lightweight autotuning
- `HELION_AUTOTUNE_EFFORT=`**`full`**, do full autotuning to **get the best config**

Shape Specialization

Current supported APIs:

- `static_shapes=True/False` in kernel setting
- `hl.specialize()` on specific tensor shape within kernel

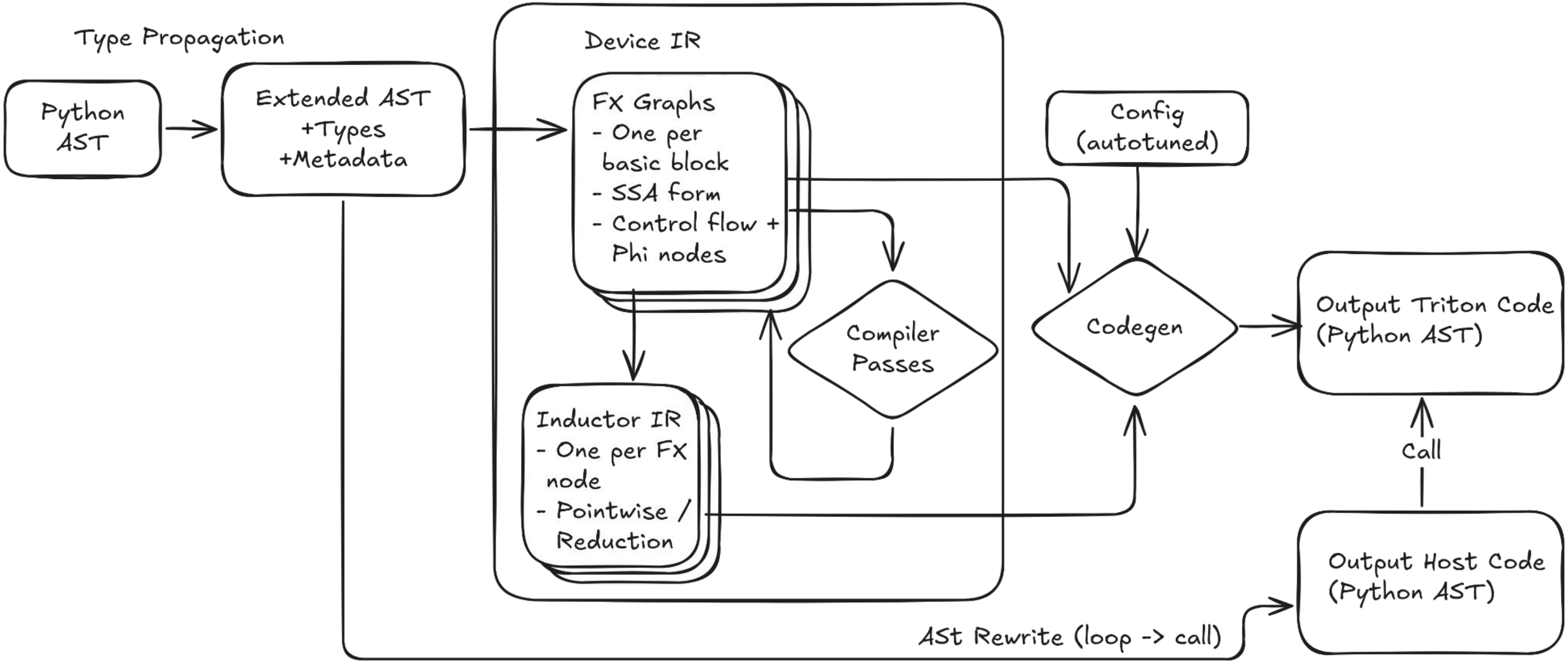
Future:

- Specialize on tensor shape outside of kernel definition (PR #1210)
- Specialize on tensor stride (PR #1215)
- Settings to avoid 0/1 specialization (PR #1053)
  - so that kernel from first `m=1` shape can still be used for `m>1` shapes

What’s happening in 2026

- Technical Roadmap
  - Distributed support
  - Automatically generate backward kernels
  - Prologue and epilogue fusions
  - New backends (Pallas and CuteDSL)
- Helion Tutorial @ PDLI 2026 (June 2026)
- Stay tuned for more events!

Compiler Internals



Try Helion Today!

- Open source contributions are welcome!
- Connect with us on GPU MODE Discord in the #helion channel.

Code: https://github.com/pytorch/helion

Docs: https://helionlang.com

Blog: https://pytorch.org/blog/helion/

Pedro Torruella

NVIDIA, Senior Software Engineer

Helion Hackathon

Pedro Torruella, SW Engineer | March 14th 2026

“

Our Mission: Great experience programming GPUs with Helion

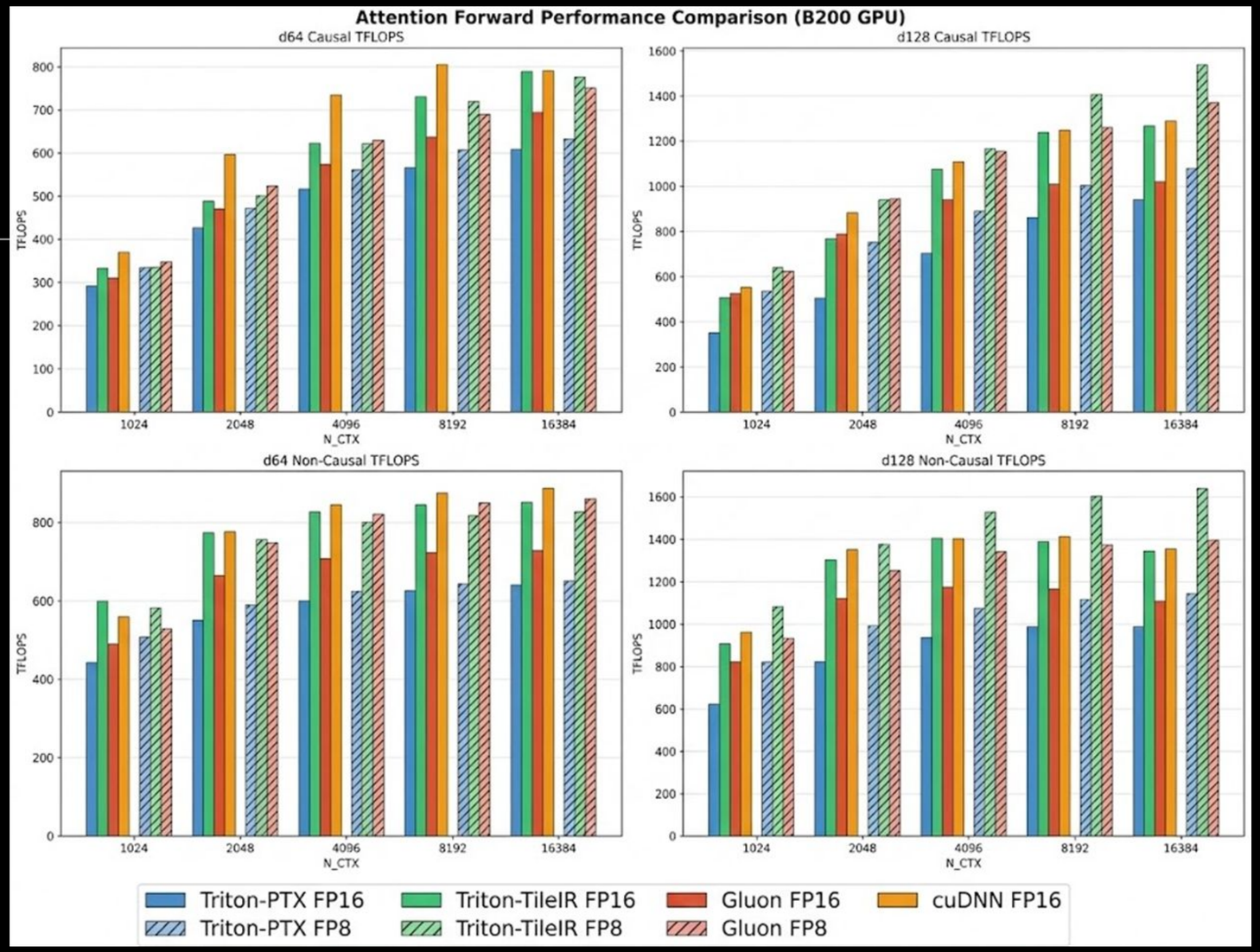
— Your Nvidia hackathon support team

”

Triton with TileIR

Advantages

- Provides optimizations like automatic pipelining, and warp speciation
- Can provide up to 1.6x speed up for certain workloads
- Graph: Tutorial's fused-attention kernel.



Triton with TileIR

Usage and Documentation

Usage

- TileIR backend is already integrated in the env
- Environment Variables:
  - `ENABLE_TILE=1 HELION_BACKEND=tileir`
- See the release page from the QR to install from a wheel

Limitations

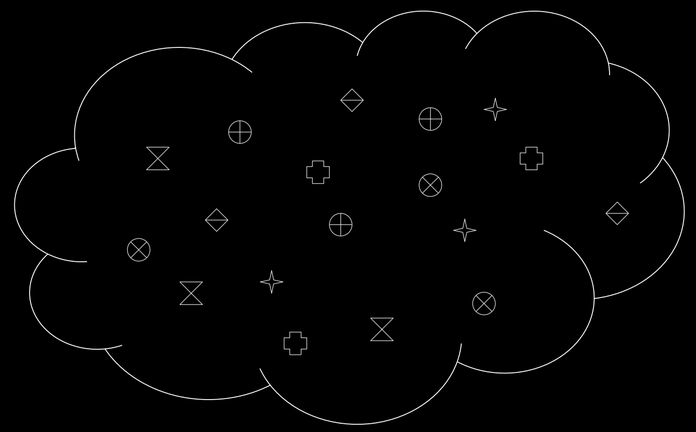
- TTIR ops are not fully supported
  - Might encounter failures during IR lowering:`inline_asm_op`
- Elementwise op might be slower than traditional backend
- Helion configs may differ from original backend

CompileIQ

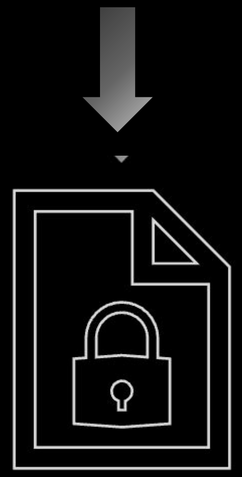
Advanced Control Files

The API to modify compilation behavior

- Contain highly tuned compiler configurations
- “-O3” for your application



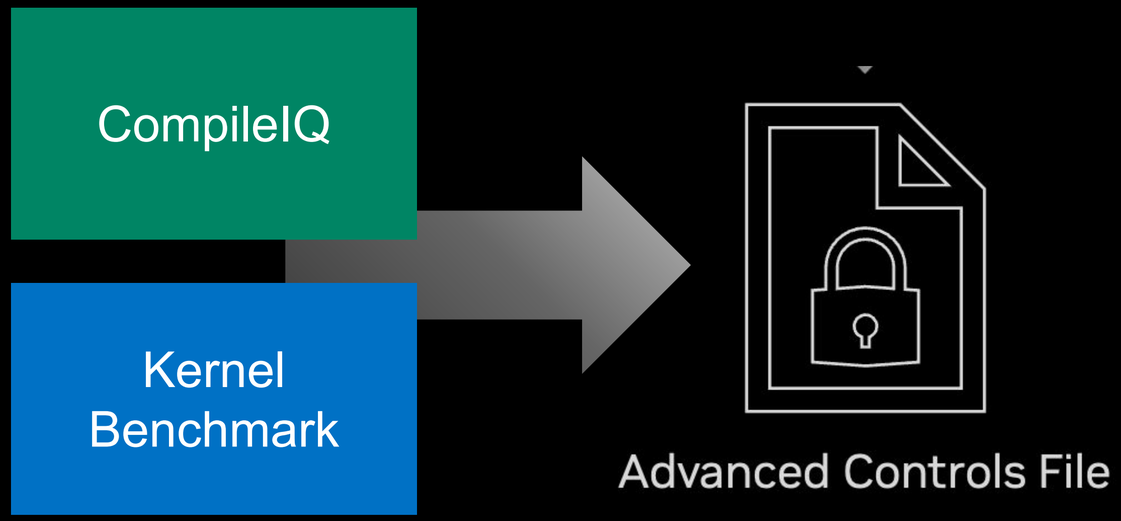
1000s of compiler settings



Advanced Controls File

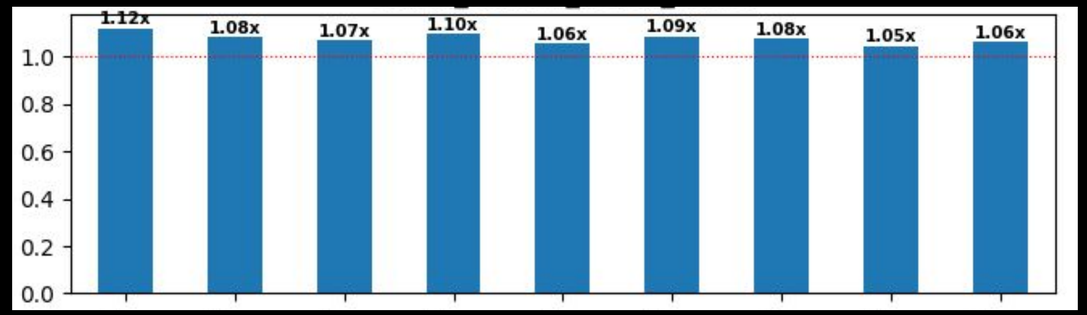
CompileIQ

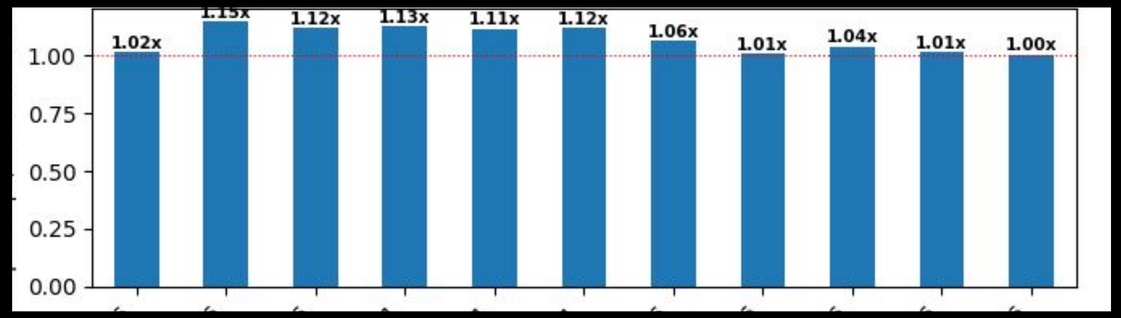
Nvidia's compiler auto-tuner for all applications



- Python Module released alongside CUDA 13.3
- Announced on GTC this Monday

ACF Gains for two hackathon example kernels





Booster Pack

Using ACFs today

- Natively supported in Helion
  - Part of **helion.Config**
  - Can select single ACF
  - Can use Helion's auto-tuner over a list of ACFs
- Available in all machines:
  - /opt/booster_pack/
- Available in submission machines

```
config=helion.Config(
        advanced_controls_file=
          "fp8_group_quant_6.acf",
        block_sizes=[4],
        num_stages=2,
        num_warps=8,
        reduction_loops=[512],
    ),
```

“

These products and features are highly experimental.

They are opt-in and not mandatory

— Your Nvidia hackathon support team

”

Come chat to us!

NVIDIA®

Hackathon Rules

Submissions Due: Saturday, March 14th, 6:00 PM
No Prior Projects: All work must be original
Team Size: Up to 4 members
Respect the Space: Treat it like Grandma’s house

Prizes

First Place

NVIDIA DGX Spark

1 NVIDIA GTC Pass (Exhibit Pass) and opportunity to demo on Meta booth

Second Place

NVIDIA GPU 5090

Third Place

NVIDIA GPU 5080

Special Prizes

Most Innovative Contribution to Helion

2 Ray-Ban Meta Glasses

2 NVIDIA GTC Passes
(1 Conference Pass + 1 Exhibit Pass) & opportunity to demo on Meta booth

Best Contribution Overall to Helion

2 Ray-Ban Meta Glasses

2 NVIDIA GTC Passes
(1 Conference Pass + 1 Exhibit Pass) & opportunity to demo on Meta booth

Best Use of NVIDIA CompileIQ

NVIDIA Gear Packs
(Judged based on the performance boost obtained from using CompileIQ)

Judges

Ujval Kapasi
VP of AI, HPC Frameworks & Libraries
NVIDIA

Luis Ceze
VP of AI Systems Software
NVIDIA

Jana van Greunen
Director of PyTorch Engineering
Meta

Horace He
Member of Technical Staff
Thinking Machines Lab

Hacker Resources

Wi-Fi